

Aplikasi Prediksi Kebutuhan Bahan Baku (ARIMA)

Dokumen ini berisi blueprint lengkap + kode starter untuk membangun aplikasi yang Anda minta: **prediksi penjualan per menu menggunakan ARIMA → konversi ke kebutuhan bahan baku → rekomendasi pengadaan**. Cocok sebagai MVP (minimum viable product) yang bisa dikembangkan.

Ringkasan Tujuan

- Input: file penjualan (CSV/Excel) periodik (mis. Jan–Des 2024)
- Output: prediksi penjualan per menu (mis. Jan–Des 2025), kebutuhan bahan baku setiap bulan, rekomendasi pengadaan, dan export (CSV/Excel)
- Metode statistik: model ARIMA (bisa pmdarima auto_arima atau statsmodels SARIMAX jika perlu musiman)
- Target pengguna: pemilik kedai/kantin yang ingin rencana stok berbasis prediksi

MVP Scope (fungsi inti)

1. Upload data penjualan (CSV / Excel)
2. Pra-proses: bersihkan, group per menu & per bulan
3. Tampilkan menu terlaris (ranking berdasarkan Qty) dan allow multi-select
4. Untuk tiap menu terpilih: fit ARIMA & generate forecast (unit per bulan)
5. Konversi forecast ke kebutuhan bahan baku berdasarkan *recipe* per menu
6. Tampilkan tabel & grafik; export ke CSV/Excel
7. Rekomendasi pengadaan (selisih kebutuhan — stok sekarang)

Format Data yang Diharapkan

Contoh CSV minimal kolom:

- `sales_date` (YYYY-MM-DD)
- `menu` (string)
- `qty` (integer)
- `nett_sales` (opsional)
- `total_after_bill_discount` (opsional)

Contoh baris:

```
sales_date,menu,qty,nett_sales,total_after_bill_discount
2024-01-03,Americano,2,40000,40000
2024-01-03,Latte,1,35000,35000
```

Data Master yang Dibutuhkan

- `recipes` (untuk tiap menu: list bahan dan jumlah per unit)
 - contoh: `Latte`: `[{ingredient: "kopi", qty: 10, unit: "gram"}, {ingredient: "susu", qty: 150, unit: "ml"}]`
 - `inventory` (stok bahan saat ini): `ingredient_id, name, qty, unit`
 - `units` & `unit_conversions` jika perlu konversi antar unit
-

Arsitektur Sistem (saran)

Backend: Python (FastAPI) — meng-handle upload, pra-proses, training ARIMA, forecasting, conversion, export.

Modeling: `pmdarima (auto_arima)` atau `statsmodels.tsa.statespace.SARIMAX` jika butuh kontrol musiman/eksojen.

Database: PostgreSQL (atau MySQL). Untuk MVP bisa pakai SQLite.

Frontend: React + Vite (TypeScript) — halaman upload, preview, pilih menu, setting ARIMA (opsional), tabel hasil, grafik.

Wadah/Deployment: Docker + docker-compose; deploy ke VPS (DigitalOcean/Hetzner) atau platform seperti Railway/Render.

Flow API (endpoints utama)

- `POST /upload-sales` — upload CSV/Excel → returns job id + parsed preview
 - `GET /menus` — list menu aggregated (qty per period)
 - `POST /forecast` — body: `{menus: ["Latte", "Americano"], periods: 12, freq: "M"}` → runs ARIMA per menu → returns forecasts
 - `POST /convert-to-ingredients` — body: `{forecast, recipes}` → returns kebutuhan bahan baku per bulan
 - `GET /inventory` — current inventory
 - `POST /recommendation` — body: `{requirements, inventory}` → returns procurement suggestions
 - `GET /export` — download CSV/XLSX
-

Algoritma (pseudocode)

1. Parse dan group: `df.groupby([menu, month]).sum(qty)` → time series per menu
2. Untuk tiap menu terpilih:
3. Pastikan minimal length (mis. ≥ 12 months). Jika kurang, fallback: simple average/seasonal naive
4. Fit `auto_arima(y, seasonal=True|False)` (uji AIC/BIC)
5. Forecast `n_periods`

6. Gabungkan hasil forecast ke dataframe per-bulan
 7. Konversi ke bahan: untuk setiap bulan, untuk setiap menu: `needed_ingredient_qty = forecast_units * recipe_qty_per_unit`
 8. Aggregate bahan per bulan untuk semua menu
 9. Rekomendasi: `buy_qty = max(0, needed - current_stock)`
-

Kode Starter — Backend (FastAPI + pmdarima)

file: `app/main.py`

```
from fastapi import FastAPI, File, UploadFile
import pandas as pd
from pydantic import BaseModel
import io
import pmdarima as pm

app = FastAPI()

class ForecastRequest(BaseModel):
    menus: list
    periods: int = 12

@app.post('/upload-sales')
async def upload_sales(file: UploadFile = File(...)):
    content = await file.read()
    df = pd.read_csv(io.BytesIO(content))
    # minimal parsing
    df['sales_date'] = pd.to_datetime(df['sales_date'])
    df['month'] = df['sales_date'].dt.to_period('M').dt.to_timestamp()
    agg = df.groupby(['menu', 'month']).agg({'qty': 'sum'}).reset_index()
    return {'preview_rows': agg.head(10).to_dict(orient='records')}

@app.post('/forecast')
def forecast(req: ForecastRequest):
    # assume sales stored/parsed in memory or DB; here simplified
    results = {}
    for menu in req.menus:
        # load series
        # y = load_series(menu) # user will implement data access
        # example fallback: naive series
        y = pd.Series([10,12,11,13,15,14,16,20,19,18,21,23],
                      index=pd.period_range('2024-01', periods=12,
                      freq='M').to_timestamp())
        model = pm.auto_arima(y, seasonal=True, m=12, error_action='ignore',
                              suppress_warnings=True)
        f = model.predict(n_periods=req.periods)
        idx = pd.period_range(y.index[-1]+pd.offsets.MonthBegin(1),
                              periods=req.periods, freq='M').to_timestamp()
        results[menu] = {'forecast': [int(x) for x in f], 'index':
```

```
[str(i.date()) for i in idx]]
    return results
```

Catatan: file ini adalah **starter**. Anda harus ganti data source `y` dengan data hasil upload / DB.

Kode Starter — Konversi Bahan (Python)

```
# input: forecasts dict {menu: {index:[..], forecast:[..]}}
# recipes: {menu: [{ingredient, qty_per_unit, unit}]}

import pandas as pd

def convert_forecast_to_ingredients(forecasts, recipes):
    rows = []
    for menu, data in forecasts.items():
        for date_str, units in zip(data['index'], data['forecast']):
            for r in recipes[menu]:
                rows.append({
                    'month': date_str,
                    'menu': menu,
                    'ingredient': r['ingredient'],
                    'qty_needed': units * r['qty_per_unit'],
                    'unit': r['unit']
                })
    return
pd.DataFrame(rows).groupby(['month', 'ingredient', 'unit']).sum().reset_index()
```

Frontend (struktur singkat)

- Halaman `Upload` — drag & drop, preview table
- Halaman `Menu Selection` — list menu terlaris (rank), checkbox multi-select
- Halaman `Forecast Config` — pilih periode (12 bulan), opsi seasonal on/off
- Halaman `Results` — tabel forecast per menu (download CSV) + tabel kebutuhan bahan + grafik (chartjs atau recharts)

Contoh endpoint fetch: `POST /forecast` lalu `POST /convert-to-ingredients` untuk menampilkan tabel kebutuhan.

UX/Visualisasi

- Untuk tiap menu: line chart forecast (actual historical + forecast)
- Untuk bahan: stacked bar per bulan (bahan teratas)
- Export ke Excel: gunakan `pandas.to_excel()` di backend atau `SheetJS` di frontend

Edge Cases & Validasi

- Data kurang (< 12 bulan): gunakan metode fallback (rolling mean, atau simple exponential smoothing)
 - Outliers besar: sediakan opsi `trim outliers` atau robust scaling
 - Non-stationary seri: auto_arima akan menangani diferensiasi, tetapi sediakan opsi manual
-

Rencana Iterasi (Roadmap)

1. Week 1: Backend minimal upload + aggregate + menu ranking + simple forecast endpoint (auto_arima) + convert-to-ingredients
 2. Week 2: Frontend upload & selection UI, results table, export
 3. Week 3: Inventory integration + procurement suggestions
 4. Week 4: Improve modeling (cross-validation, interval forecast, seasonal tuning), user auth, deploy
-

Deliverables Yang Saya Siapkan Jika Anda Mau Saya Bantu Implementasi

- Repo GitHub berisi backend FastAPI + contoh data + frontend React (TS) starter
 - Notebook Jupyter yang menjelaskan pipeline ARIMA (EDA, fit, diagnostics)
 - Dockerfile & docker-compose untuk testing lokal
-

Langkah Selanjutnya (Pilihan untuk Anda)

1. Saya buat repo starter (FastAPI + React) dengan fitur: upload, aggregate, forecast (dummy data) — lalu Anda jalankan lokal.
 2. Anda unggah contoh CSV penjualan (Januari–Desember 2024) supaya saya siapkan notebook dan endpoint yang langsung bisa memproses data nyata.
 3. Saya langsung buat notebook Jupyter yang memuat pipeline ARIMA & contoh konversi bahan, lalu Anda jalankan.
-

Jika Anda setuju, pilih salah satu langkah di atas (1, 2, atau 3). Kalau mau, saya akan mulai membuat repo starter berisi kode yang saya sertakan di dokumen ini.